PRODUCT REQUIREMENTS DOCUMENT · V1.0

KITOS 2.0: Kids In Tech LMS on MERN

Keep the existing Next.js interface; replace the backend with Express, Node.js and MongoDB Atlas on Render; add the Kids In Tech logo and StarNova Labs attribution

PRODUCT OWNER	Aliyu Mubarak, Founder and CEO, StarNova Labs
PRODUCT	KITOS (Kids In Tech Operating System)
NEW REPOSITORY	`Kids In Tech\KITOS-MERN\`
REFERENCE CODE	`kit-lms-main` (frontend, PHP API, schema.sql)
HOSTING	Vercel (client) · Render (API) · MongoDB Atlas (data)
DATE	17 October 2026

Build instructions for Claude Code are in docs/BATCH-01-PROMPT.md to BATCH-03-PROMPT.md. This PRD is the source of truth.

A STARNOVA LABS PROJECT

STARNOVA

www.starnovalabs.com · kidsintech.school

CONTENTS

1. Document Control....1
2. Summary....1
3. Background and Problem....2
3.1 Kids In Tech....2
3.2 Current state audit....2
3.3 Problem statement....3
4. Goals, Non-Goals and Success Metrics....3
4.1 Goals....3
4.2 Non-goals....3
4.3 Success metrics....3
5. Users and Roles....4
6. Scope: Functional Requirements....4
7. Architecture....5
7.1 Overview....5
7.2 Repository layout....5
7.3 Technology....6
7.4 Conventions....6
8. Data Model....7
8.1 Legacy schema (for reference)....7
8.2 Collections....7
9. API Specification....9
9.1 Auth and account (Batch 1)....9
9.2 Users, parents, students, instructors (Batch 1)....9
9.3 Learning content (Batch 2)....10
9.4 Sessions and attendance (Batch 2, new)....10
9.5 Chat, notifications, payments, admin (Batch 3)....11
10. Security and Child Safety....11
10.1 Permission matrix (defaults)....12
11. Integrations....13
11.1 Paystack (test mode)....13
11.2 Cloudinary....13
11.3 Email....13
12. Frontend Changes (only these)....13
13. Non-Functional Requirements....14
13.1 Environment variables....14
14. Acceptance Criteria (release)....15
15. Execution Plan....16
15.1 Batches....16
15.2 Deployment steps (summary)....16
15.3 GoMyCode checkpoint coverage....16
16. Risks and Mitigations....17
17. Open Questions....17

1. Document Control

Item	Detail
Product	KITOS 2.0 (Kids In Tech Operating System)
Document	Product Requirements Document (PRD)
Version	1.0, 17 October 2026
Owner	Aliyu Mubarak, Founder and CEO, StarNova Labs
Programme context	GoMyCode MSc Software Engineering, checkpoint "Cloud-Based Web System Design"
Repository	`Kids In Tech\KITOS-MERN\` (new); GitHub `MubarakAliyu/KITOS-MERN` (to be created)
Reference code (read-only)	`Kids In Tech\kit-lms-main\kit-lms-main\` (frontend, PHP API, SQL schema)
Builder	Claude Code, one batch at a time, from `docs/BATCH-0N-PROMPT.md`
Status	Approved for build

How to read this document. Sections 2 to 5 explain why and what. Sections 6 to 13 are the build specification. Section 14 sets acceptance criteria and Section 15 the execution plan. Where this PRD and the batch prompts disagree, this PRD wins; open questions go to `docs/DECISIONS.md`.

2. Summary

KITOS is the learning management system that supports Kids In Tech's online sessions alongside its physical coding bootcamps for children aged 8 to 18. Its frontend is complete: a Next.js 16 / React 19 application with dashboards for students, parents, instructors and administrators. But it runs entirely on mock data (Mock Service Worker), and the backend that exists, a PHP/MySQL API, is not connected, cannot be deployed safely and disagrees with the frontend on field names.

KITOS 2.0 keeps the frontend exactly as designed and replaces the backend with a MERN API Node.js 22, Express 5 and MongoDB Atlas, deployed on Render, with the frontend deployed on Vercel. The only visible changes to the interface are the Kids In Tech logo in place of the lightning-bolt icon, and a "Powered by StarNova Labs" attribution with the StarNova Labs logo at the bottom of the sidebar and the sign-in page.

3. Background and Problem

3.1 Kids In Tech

Kids In Tech (KIT) is a STEM programme by StarNova Labs, running physical bootcamps in Birnin Kebbi and Sokoto. Three cohorts have completed and more than 111 students have been trained. The learning path branches: Scratch Programming first, then Web Development or Robotics in parallel, then an Advanced Track. KIT's policy requires positive, encouraging language toward every child.

3.2 Current state audit

Asset	Location	State	Decision
Frontend	`kit-lms-main/src`	Next.js 16.2, React 19.2, JavaScript, Tailwind v4, NextAuth (Credentials), Zustand, Axios, socket.io-client, react-hook-form + zod, Recharts, Sonner, @react-pdf/renderer, next-themes, EN/HA locales. About 60 screens across four roles.	**Keep.** Copy into `KITOS-MERN/client/`. Do not redesign.
API contract	`src/_lib/api/*.js`, `src/_lib/mocks/handlers.js` (2,124 lines), `src/_types/schema.js`	About 70 endpoints called by the frontend, with full response shapes in the mock handlers.	**Source of truth** for the new API.
Live sessions	`src/_lib/api/sessions.js`	Mocked inside the client (no API call): list, detail, create, update, cancel, reschedule, attendance roster, save attendance, recordings, admin overview and attendance report.	Build these endpoints; point the service at the API.
PHP API	`kit-lms-main/api/index.php` (3,579 lines), `auth.php`, `config.php`, `db.php`	About 115 routes in one file. JWT signature verification disabled; hard-coded secret and DB credentials; placeholder refresh token; "must reset" detected by comparing with a default password; DB errors leaked to clients.	**Do not port code.** Read for business rules only.
SQL schema	`kit-lms-main/api/schema.sql`	15 MySQL tables (see Section 8.1). No enrolments table; quiz options stored as JSON text; settings in a JSON file.	Map to MongoDB collections (Section 8).
Seed data	`api/seed.php`, `api/seed_full.php`, mock handlers	Demo accounts and fixtures.	Recreate in `server/src/seed/`.
FastAPI prototype	`Kids In Tech/LMS/kit-lms`	Earlier AI-generated build, MongoDB, partial.	Ignore except as a reference for collection ideas.

3.3 Problem statement

Learning stops when a bootcamp ends; online sessions run on scattered tools; parents have no single view of progress, attendance and payments; children's data is handled without enforced consent (as the Nigeria Data Protection Act 2023 requires); and the existing LMS cannot be deployed because it has no working, secure backend.

4. Goals, Non-Goals and Success Metrics

4.1 Goals

1. Every screen in the existing frontend works against a real, persistent backend: no mock data in production.
2. A secure Express + MongoDB Atlas API that implements the frontend's contract exactly, plus the missing sessions, attendance, recordings, consent and certificate-verification endpoints.
3. Real-time chat and notifications through Socket.IO, with child-safety rules enforced on the server.
4. Paystack payments verified server-side (test mode).
5. Kids In Tech logo and "Powered by StarNova Labs" attribution added.
6. Deployed publicly: frontend on Vercel, API on Render, database on MongoDB Atlas, with CI.
7. Satisfies every instruction of the GoMyCode checkpoint (Section 15.3).

4.2 Non-goals

- Visual redesign, new colours or new fonts.
- Rewriting the frontend in TypeScript or another framework.
- Native mobile apps, built-in video streaming, AI tutor, partner-school portal, production payment settlement.

4.3 Success metrics

Metric	Target
Frontend endpoints served by the real API	100% (no MSW in production build)
Production test checklist	All rows pass on the live URL
API tests	All green; service coverage of 70% or more
Lighthouse on live URL (login, student dashboard, lesson)	Performance 85+ mobile, accessibility 95+
API latency after warm-up	p95 under 500 ms
Unauthorised access in permission tests	Zero

5. Users and Roles

Role	Signs in with	Main jobs
Admin (`admin`)	Email	Users, courses, payments, announcements, analytics, permissions, activity log, chat monitor, sessions overview, platform settings
Instructor (`instructor`)	Email	Courses, modules, lessons, quizzes, assignments, students, submissions review, sessions, attendance, recordings, chat
Parent (`parent`)	Email	Children, consent, progress, payments, subscriptions, receipts, support chat, notifications
Student (`student`)	Admission number `KIT/YY/NNN`	Courses, lessons, quizzes, assignments, sessions, recordings, certificates, chat with instructors

Demo accounts used by the current frontend (recreate in the seed, but passwords for production demo are supplied at seed time, never committed): `admin@kidsintech.school`, `instructor@kidsintech.school`, `parent@kidsintech.school`, student `KIT/26/001`, force-reset demo `amina@gmail.com`.

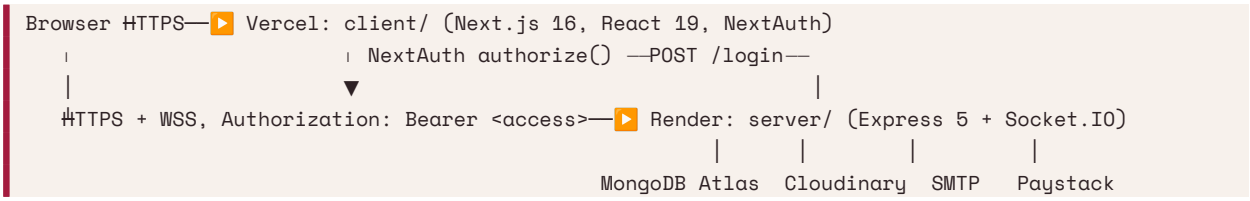
6. Scope: Functional Requirements

ID	Requirement	Batch
FR1	Sign in by email or admission number; `{ user, access }` response; account disabled check; rate-limited	1
FR2	Forced first-login reset for admin-provisioned accounts (`must_reset_password` flag, not password comparison)	1
FR3	Forgot and reset password by emailed single-use token (hashed, 30-minute expiry)	1
FR4	Change password, update profile, language preference	1
FR5	Admin creates users of any role; admission number auto-generated atomically; activate, deactivate, assign instructor; permissions matrix editable	1
FR6	Parent adds a child (name, age 8–18, programme track) and accepts the consent notice; child account is pending until consent is recorded	1
FR7	Courses, modules, lessons (text, YouTube, resources), publish and unpublish, full admin course editor	2
FR8	Quizzes with embedded questions; auto-graded submission; one attempt record per student per quiz (retakes update best score)	2
FR9	Assignments; student submission (text, link or file); instructor feedback and grade; status pending, submitted, reviewed	2

ID	Requirement	Batch
FR10	Enrolment (explicit collection); lesson completion; progress recalculated when content is added or removed	2
FR11	Certificates issued on 100% completion with serial and verification code; public verification endpoint	2
FR12	Sessions: create, update, reschedule, cancel (with reason); physical, online or hybrid; computed status scheduled, live, completed, cancelled	2
FR13	Attendance: roster save by instructor; QR check-in by student with a signed, 60-second token; admin attendance report	2
FR14	Recordings attached to sessions; visible to enrolled students	2
FR15	Chat over Socket.IO and REST: conversations, messages, read receipts, typing; child-safety rules (Section 10)	3
FR16	Paystack initiate, verify and webhook; receipts `RCP-YYYYMM-NNNN`; subscriptions and cancellation; parent and admin payment views with CSV	3
FR17	Notifications (persisted, real-time push) and admin announcements with fan-out	3
FR18	Admin stats, analytics (enrolment trend, revenue, completion, attendance), pending enrolments, activity log	3
FR19	Platform settings stored in the database (`GET/POST /platform/settings`)	3
FR20	Kids In Tech logo replaces the bolt icon (sidebar, collapsed sidebar, sign-in, favicon, certificate PDF); "Powered by StarNova Labs" with logo at the bottom of the sidebar and sign-in page, light and dark variants, linking to https://www.starnovalabs.com	1

7. Architecture

7.1 Overview



- The browser calls the API directly at `NEXT_PUBLIC_API_URL` (the Render URL or `api.kidsintech.school`). CORS allows only `CLIENT_ORIGIN`. No cookies are used by the API; authentication is the Bearer token that NextAuth keeps in its encrypted session cookie.
- Socket.IO runs on the same Render service and port, authenticated with the same token in `auth.token`.

7.2 Repository layout



```
├── server/
│   ├── package.json          "type": "module", Node 22
│   ├── src/
│   │   ├── app.js            express app factory (testable)
│   │   ├── index.js          connect DB, create HTTP server, attach Socket.IO, listen
│   │   ├── config/           env.js (zod), db.js, permissions.js, constants.js
│   │   ├── models/           one file per collection
│   │   ├── routes/           auth, users, admin, parents, students, instructors, courses,
│   │   │                     modules, lessons, quizzes, assignments, sessions, certificates,
│   │   │                     payments, chat, notifications, settings, health
│   │   ├── controllers/
│   │   ├── services/         all business rules live here
│   │   ├── middleware/       authenticate, requirePermission, validate, rateLimit, error
│   │   ├── realtime/         socket.js (auth, rooms, join/message/typing/notification)
│   │   ├── jobs/             reminders.js (interval + run on boot)
│   │   ├── lib/              tokens, numbering, pagination, mailer, storage, paystack, pdf,
serialize
│   │   └── seed/             seed.js (refuses in production unless ALLOW_DEMO_SEED=true)
│   └── test/                 vitest + supertest + mongodb-memory-server
├── docs/                     PRD.md, CLAUDE.md copy, LAB-PHASES.md, BATCH-0N-PROMPT.md,
│                             API.md, DEPLOYMENT.md, DECISIONS.md, PROGRESS.md, DEMO-SCRIPT.md
├── render.yaml
├── .github/workflows/ci.yml
├── CLAUDE.md
-- README.md
```

7.3 Technology

Concern	Choice
Runtime	Node.js 22 LTS, npm (each package has its own lockfile)
API	Express 5, `cors`, `helmet`, `express-rate-limit`, `pino-http`, `express-mongo-sanitize` equivalent (custom key sanitiser for Express 5)
Data	Mongoose 8 on MongoDB Atlas M0 (replica set, transactions)
Validation	Zod
Auth	`jsonwebtoken` (HS256, 24 h, includes `tv` token version), `bcryptjs` cost 12
Real-time	Socket.IO 4
Files	Cloudinary (`STORAGE_PROVIDER=cloudinary`), local disk in development
Email	Nodemailer over SMTP; writes to `server/.local/outbox/` when SMTP is unset
PDF	`pdfkit` on the server for certificates and receipts (the client's @react-pdf viewer stays)
Tests	Vitest, Supertest, mongodb-memory-server (replica set), Playwright for E2E against the client
Lint	ESLint 9 (client keeps `eslint-config-next`)

7.4 Conventions

- **JSON shape matches the frontend**: snake_case fields, `id` (string) instead of `_id`, ISO 8601 dates. A shared `toJSON` transform on every schema does this.

- **Layers**: route → controller → service → model. Controllers contain no business logic. Services never read `req`.
- **Errors**: `{ error: "<message>", reason?: "<detail>", fields?: { name: msg } }` with correct HTTP status. The frontend interceptor reads `error` and `reason`.
- **Money**: integer kobo in the database; the API returns naira numbers where the current frontend expects naira (`amount: 15000`), converting at the serialiser. Recorded as an ADR.
- **Time**: stored in UTC; displayed by the client in Africa/Lagos.
- **IDs**: MongoDB ObjectIds as strings. The frontend never parses ids as numbers (verify and fix any `Number(id)` usage in Batch 1).

8. Data Model

8.1 Legacy schema (for reference)

`users`, `parents`, `students`, `courses`, `modules`, `lessons`, `quizzes`, `questions`, `assignments`, `submissions`, `payments`, `messages`, `notifications`, `lesson_completions`, `quiz_submissions`. Settings lived in `api/settings.json`.

8.2 Collections

Collection	Fields (all have `created_at`, `updated_at`)	Indexes
users	`role` enum, `email` (lowercase, sparse unique), `admission_no` (sparse unique), `name`, `password_hash`, `is_active`, `language_preference` en/ha, `must_reset_password`, `token_version`, `avatar_url`, `last_login_at`	email, admission_no, role
students	`user_id`, `parent_id` (user), `name`, `age`, `programme_track` enum scratch/web-development/robotics/advanced, `cohort_ids[]`, `consent_id`, `status` pending/active	user_id unique, parent_id
consents	`parent_id`, `student_id`, `version`, `scopes[]`, `accepted_at`, `ip`, `user_agent`	student_id
cohorts	`name`, `programme_track`, `start_date`, `end_date`, `venue`, `mode`, `instructor_ids[]`	programme_track
courses	`title`, `description`, `thumbnail`, `instructor_id`, `programme_track`, `status` draft/published, `price_kobo`, `payment_type` subscription/one_time	status, instructor_id
modules	`course_id`, `title`, `order_index`	course_id+order_index
lessons	`module_id`, `course_id`, `title`, `content_type` video/text, `youtube_url`, `body_content` (sanitised HTML), `resources[]` `{title,url}`, `order_index`, `duration_minutes`	module_id+order_index
quizzes	`module_id`, `course_id`, `lesson_id?`, `title`, `order_index`, `questions[]` `{question_text, question_type` `mcq/true_false, options[], correct_answer}`	module_id

Collection	Fields (all have created_at, updated_at)	Indexes
quiz_attempts	quiz_id, student_id, answers[], score, total, percentage, attempts	quiz_id+student_id unique
assignments	module_id, course_id, title, instructions, deadline	module_id
submissions	assignment_id, student_id, submission_type text/link/file, content, file_url, status submitted/reviewed, grade, feedback, reviewed_by, reviewed_at	assignment_id+student_id unique
enrollments	student_id, course_id, status pending/active/completed/cancelled, progress 0–100, source admin/payment, completed_at	student_id+course_id unique
lesson_completions	student_id, lesson_id, course_id, completed_at	student_id+lesson_id unique
certificates	student_id, course_id, serial KIT-CERT-YYYY-NNNN, verification_code (10 chars), issued_at, pdf_url	serial unique, verification_code unique
sessions	title, description, course_id, cohort_id?, instructor_id, scheduled_at, duration_minutes, mode online/physical/hybrid, meeting_url, venue, status scheduled/cancelled, cancel_reason, rescheduled_from	scheduled_at, course_id
attendance	session_id, student_id, status present/absent/late/excused, method roster/qr/auto, marked_by, marked_at	session_id+student_id unique
recordings	session_id, title, url, source youtube/drive/other, added_by	session_id
payments	parent_id, student_id, course_id, amount_kobo, type, status pending/success/failed, paystack_ref unique, receipt_no, paid_at, channel	parent_id, status
subscriptions	parent_id, student_id, course_id, status active/cancelled, next_billing_at, cancelled_at	parent_id
conversations	participants[] {user_id, role}, type student_instructor/parent_instructor/parent_admin, last_message_at, observer_ids[] (parents of student participants)	participants.user_id
messages	conversation_id, sender_id, receiver_id, message, file_url, is_read, reported	conversation_id+created_at
notifications	user_id, type, title, message, link, is_read	user_id+is_read
announcements	title, message, audience all/role/course, target, sent_by	created_at
activity_logs	actor_id, actor_role, action, target_type, target_id, meta, ip (append-only)	created_at
settings	singleton: platform_name, welcome_message, primary_color, logo_url, permissions matrix overrides	—
counters	_id key (admission:26, receipt:202610, cert:2026), seq	—

9. API Specification

Base URL: `NEXT_PUBLIC_API_URL`. All routes except those marked Public require `Authorization: Bearer <access>`. The path list below is the frontend's contract; match paths, methods and response shapes in `client/src/_lib/mocks/handlers.js` exactly.

9.1 Auth and account (Batch 1)

Method	Path	Notes
POST	/login	Body `{ email, password }` where `email` may be an admission number. Returns `{ access, user }` or `{ must_reset_password: true, temp_token, user }`. 401 `{ error: "Invalid credentials" }`.
POST	/forgot-password	Always 200 (no user enumeration); emails reset link to `${CLIENT_ORIGIN}/reset-password?token=`
POST	/reset-password	`{ token, password }` or forced reset with `temp_token`
POST	/user/change-password	Increments `token_version`
PUT	/user/profile	Name, avatar (students: read-only fields per frontend copy)
PUT	/user/language	`en` or `ha`
GET	/health	Public. `{ ok: true, db: "up" }`

9.2 Users, parents, students, instructors (Batch 1)

Method	Path	Notes
GET, POST	/admin/users	List with filters; create any role (student requires parent and track; generates admission number; `must_reset_password: true`)
PUT	/admin/users/:id	Update
POST	/admin/users/activate, /admin/users/deactivate	Deactivate increments `token_version`
POST	/admin/users/assign-instructor	Assign instructor to course
GET	/admin/next-admission-no	Preview next number (not reserved)
POST	/admin/permissions	Save matrix overrides to settings
GET	/parents/me, /parents/me/children	Children with progress summary
POST	/students	Parent adds child with `consent: { version, accepted: true }`
GET	/students/me	Student profile and enrolments

Method	Path	Notes
GET	/students/:id/progress	Parent (own child), instructor (own student), admin
GET	/instructors/me, /instructors/me/courses, /instructors/me/students	Instructor views

9.3 Learning content (Batch 2)

Method	Path
GET	/courses, /courses/:id/modules
POST	/courses, /modules, /lessons, /quizzes, /assignments
GET, PUT, DELETE	/lessons/:id; DELETE /modules/:id; PUT, DELETE /quizzes/:id; PUT /assignments/:id
GET	/modules/:id/lessons, /quizzes/:id/questions (without `correct_answer` for students)
POST	/lessons/:id/complete, /quiz/submit
GET	/assignments (student), /assignments/instructor
POST	/assignments/submit; PUT /assignments/:id/feedback
GET, POST	/admin/courses; PUT, DELETE /admin/courses/:id; POST /admin/courses/:id/publish, /unpublish; GET /admin/courses/:id/editor
GET	/certificates (own); GET /certificates/:id/pdf; GET /certificates/verify/:code (Public)

9.4 Sessions and attendance (Batch 2, new)

Method	Path	Replaces client function
GET	/sessions?filter=all|live|upcoming|completed	getSessions, getLiveSessions, getUpcomingSessions, getCompletedSessions
GET	/sessions/:id	getSession, getInstructorSession
GET	/instructors/me/sessions	getInstructorSessions
POST	/sessions	createSession
PUT	/sessions/:id	updateSession
POST	/sessions/:id/cancel	cancelSession { `reason` }
POST	/sessions/:id/reschedule	rescheduleSession { `scheduled_at` }
GET	/sessions/:id/roster	getAttendanceRoster
PUT	/sessions/:id/attendance	saveAttendance { `records: [{ student_id, status }]` }

Method	Path	Replaces client function
GET	/sessions/:id/attendance	getSessionAttendance
POST	/sessions/:id/qr-token; POST /sessions/:id/check-in	New: instructor shows QR; student checks in
POST	/sessions/:id/recordings	addRecording { url, source }
GET	/sessions/cancellations	getSessionCancellations
GET	/admin/sessions/overview, /admin/sessions, /admin/sessions/attendance-report	getAdminSessionsOverview, getAdminAllSessions, getAdminAttendanceReport

Status is computed at read time from `scheduled_at` and `duration_minutes` exactly as `computeStatus()` in the current client mock does.

9.5 Chat, notifications, payments, admin (Batch 3)

Method	Path
GET	/conversations; GET /messages?conversation_id=; POST /messages; PUT /messages/read
GET	/admin/chat/all-conversations
GET	/notifications; PUT /notifications/:id/read; PUT /notifications/read-all
POST	/admin/notifications/send (announcement + fan-out)
POST	/paystack/initiate; POST /paystack/verify; POST /paystack/webhook (Public, HMAC)
GET	/payments; GET /payments/stats; GET /payments/:id/receipt
GET	/subscriptions; POST /subscriptions/cancel
GET	/admin/stats, /admin/analytics, /admin/payments, /admin/pending-enrollments, /admin/activity-log
GET, POST	/platform/settings

Socket.IO events (server implements what `useChat.js` emits): client → server `join { conversation_id }`, `message { conversation_id, message, file_url? }`, `typing { conversation_id }`; server → client `message`, `typing`, `notification`, `session:live`.

10. Security and Child Safety

Control	Rule
Authentication	JWT HS256 signed with `JWT_SECRET` (32+ bytes), verified on every request, 24 h expiry, token version check. **Never** accept unsigned tokens (the legacy defect).

Control	Rule
Passwords	bcrypt cost 12; minimum 8 characters with a letter and a number; reset tokens SHA-256 hashed, single use, 30 minutes.
Rate limits	`/login` 10 per 15 min per IP + identifier; `/forgot-password` 5 per hour; messages 30 per minute per user.
Authorisation	`requirePermission(key)` from one matrix (`config/permissions.js`, overridable from settings) **and** ownership checks in services.
Input	Zod on body, params and query; Mongo operator keys stripped; rich text sanitised with an allow-list (`sanitize-html`).
Headers and CORS	helmet; CORS only `CLIENT_ORIGIN`; JSON limit 1 MB; uploads 10 MB.
Secrets	Only in environment variables; `.env*` git-ignored; `.env.example` lists names only. The `mongoDB-credentials.txt` file in the KIT folder is never copied into the repository.
Errors	No stack traces or DB messages in production responses.
Audit	Every admin mutation and security event (login failure bursts, deactivation, permission change, payment confirmation) writes `activity_logs`.
Consent	Child account `status: pending` until parent consent recorded; pending students cannot sign in.
Child data	No email required for students; profiles private; no third-party scripts in student area.
Messaging	Allowed pairs only: student↔instructor (of an enrolled course), parent↔instructor, parent↔admin, admin↔anyone. **No student↔student.** Parents of a student are observers on that student's conversations. Admin can read all.
Positive language	Feedback endpoint checks text against a small discouraging-phrase list and returns 422 with suggestions; templates offered in the UI copy.

10.1 Permission matrix (defaults)

Permission	admin	instructor	parent	student
users.manage	✔			
course.create, course.publish	✔			
course.edit	✔	✔ (assigned)		
content.read	✔	✔	✔ (child's)	✔ (enrolled)
lesson.complete, quiz.submit, assignment.submit				✔
submission.review	✔	✔ (assigned)		
session.manage, attendance.mark, recording.add	✔	✔ (assigned)		
session.view	✔	✔	✔	✔
child.manage, payment.make			✔	

Permission	admin	instructor	parent	student
payment.view	✔		✔ (own)	
chat.use	✔	✔	✔	✔
chat.monitor, analytics.view, settings.manage, audit.view	✔			

11. Integrations

11.1 Paystack (test mode)

1. `POST /paystack/initiate { student_id, course_id, type }` → server creates `payments` (pending, ref `KIT-<ulid>`) and calls Paystack `transaction/initialize` (amount in kobo, parent email, callback). Returns `{ reference, authorization_url, access_code }`. If `PAYSTACK_SECRET_KEY` is unset, returns a simulated reference so the existing four-step modal still works in development.
2. Client completes popup, then `POST /paystack/verify { reference }` → server calls `transaction/verify/:reference`.
3. `POST /paystack/webhook` verifies `x-paystack-signature` (HMAC-SHA512 of raw body with secret key).
4. Success from either path runs one idempotent transaction: payment `success`, `receipt_no`, enrolment `active` (create if missing), subscription upsert for `subscription` type, notification to parent and admin, audit row.

11.2 Cloudinary

Uploads for submissions, lesson resources, chat attachments and certificate PDFs via a `storage` interface with `local` and `cloudinary` providers. API refuses to start if `STORAGE_PROVIDER=cloudinary` and `CLOUDINARY_URL` is missing.

11.3 Email

Password reset, child account created, payment receipt, session reminders, announcement copies. Sender `MAIL_FROM` (KIT address `hello@kidsintech.school` once verified).

12. Frontend Changes (only these)

1. **Copy** `kit-lms-main/kit-lms-main` into `KITOS-MERN/client` excluding `node_modules`, `.next`, `api/`, `.env.local`, `.claude/`.
2. **Brand assets** to `client/public/brand/`: `kit-logo.png` (full logotype, from `Kids In Tech/Branding/kids logo.png`, optimised to WebP/PNG at 2×), `kit-mark.png` (square crop for collapsed sidebar and favicon), `starnova-logo-dark.svg` (from `StarNova`

Labs/Branding/new-logo/svg/StarNova-1.svg, dark text for light theme), starnova-logo-light.svg (from StarNova.svg, white text for dark theme).

3. **Logo**: replace the bolt icon in _components/auth/BrandLockup.jsx and _components/layout/Sidebar.jsx with a <KitLogo /> component; update app/favicon.ico and the certificate template.
4. **Powered by**: new _components/brand/PoweredByStarNova.jsx — small caps "Powered by" label (Space Mono, 10–11 px, --text-muted) above the StarNova logo (height 16–18 px), theme-aware, <a href="https://www.starnovalabs.com" target="_blank" rel="noopener">, aria-label="Powered by StarNova Labs". Placed at the very bottom of the sidebar (below the user card and logout; hidden or mark-only when collapsed) and centred at the foot of the sign-in left panel.
5. **API wiring**: client.js timeout 8 s → 20 s on first request (Render cold start), keep retry; sessions.js mock bodies replaced with apiClient calls; messages.js and notifications.js completed; socket.js uses session.accessToken and NEXT_PUBLIC_SOCKET_URL.
6. **MSW**: only when NEXT_PUBLIC_API_MOCKING=enabled (default off); never in production. instrumentation.js guarded the same way.
7. No other visual changes. Fix only bugs exposed by real data (empty states, id types, date parsing).

13. Non-Functional Requirements

ID	Requirement
NFR1	Performance: Lighthouse performance 85+ (mobile) on live URL; API p95 under 500 ms after warm-up; list endpoints paginated (default 20, max 100); indexes on every query path.
NFR2	Accessibility: WCAG 2.2 AA on retained UI; axe no serious/critical; logos have alt text; new links have visible focus.
NFR3	Security: Section 10; npm audit no high/critical in production deps.
NFR4	Reliability: /health checks DB; jobs catch up on boot; idempotent payment confirmation.
NFR5	Maintainability: layered code; ESLint clean; tests for every service; docs/API.md complete.
NFR6	Portability: runs locally with npm run db:local (in-memory replica set) and no external accounts.
NFR7	Observability: pino JSON logs with request id; errors logged with stack server-side only.

13.1 Environment variables

Variable	Where	Purpose
NODE_ENV, PORT	server	production, 10000 on Render
MONGODB_URI, MONGODB_DB_NAME	server	Atlas connection, kitos
JWT_SECRET	server	32+ random bytes

Variable	Where	Purpose
CLIENT_ORIGIN	server	Vercel URL (CORS, socket, email links)
STORAGE_PROVIDER, CLOUDINARY_URL	server	`local` or `cloudinary`
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, MAIL_FROM	server	Email
PAYSTACK_SECRET_KEY	server	Test secret key
ALLOW_DEMO_SEED, DEMO_PASSWORD_*	seed run only	Never set on Render permanently
NEXT_PUBLIC_API_URL, NEXT_PUBLIC_SOCKET_URL	client	Render URL
NEXTAUTH_URL, NEXT_PUBLIC_LMS_URL, NEXTAUTH_SECRET	client	Vercel URL; session secret
NEXT_PUBLIC_PAYSTACK_PUBLIC_KEY	client	Test public key
NEXT_PUBLIC_API_MOCKING	client	`disabled` in production

14. Acceptance Criteria (release)

1. Each of the four demo roles signs in on the live URL and every sidebar item loads real data without console errors.
2. A parent adds a child, accepts consent, the child signs in with the generated admission number, and the forced reset works.
3. An instructor creates a module, lesson and quiz; the student completes them; progress updates on the parent dashboard; a certificate issues at 100% and verifies publicly.
4. An instructor schedules an online session; it shows as live at its time; attendance is saved; a recording is added and visible to the student.
5. A parent pays in Paystack test mode; enrolment activates; receipt prints; admin payments table and analytics reflect it.
6. Student and instructor chat in real time; a student cannot start a chat with another student (API returns 403); the parent can read the child's conversation.
7. KIT logo appears in the sidebar, sign-in page and favicon; "Powered by StarNova Labs" with logo appears at the bottom of the sidebar and sign-in page in both themes.
8. CI green; API tests green; Lighthouse and axe targets met; production test checklist complete in `docs/DEPLOYMENT.md`.
9. No secrets in the repository; MSW absent from the production bundle.

15. Execution Plan

15.1 Batches

Batch	Name	Contents	Lab alignment
1	Backend foundation, identity and brand	New repo, copy client, server scaffold, env, DB, health, models (users, students, consents, counters, settings, activity_logs), auth endpoints, permissions, admin users, parent/child + consent, students/me, instructors/me, KIT logo + Powered by StarNova, MSW off, seed, CI	Design architecture; develop (clean, modular); CI
2	Learning core and hybrid sessions	Courses, modules, lessons, quizzes, assignments, submissions, enrolments, progress, certificates, sessions, attendance, QR check-in, recordings, reminders job, dashboards on real data	Develop and implement
3	Real-time, payments, admin and deployment	Socket.IO chat with safety rules, notifications, announcements, Paystack, subscriptions, receipts, admin stats/analytics/payments/activity log, settings, full tests, Lighthouse/axe, render.yaml, Vercel, Atlas, production checklist, API.md, DEPLOYMENT.md, demo script	Deploy to cloud; test; document and present

Each batch: work on branch `batch/0N-…`; finish with lint, tests and both builds green; update `docs/PROGRESS.md`; report; stop.

15.2 Deployment steps (summary)

Atlas M0 (Frankfurt) → Cloudinary → SMTP (Brevo or Gmail app password) → Paystack test keys → Render Blueprint from `render.yaml` (root `server`), check `/health` → Vercel project (root `client`) with client env → set `CLIENT_ORIGIN` on Render, redeploy → register Paystack webhook → run guarded seed from a laptop → run the production test checklist → optional domain `lms.kidsintech.school`.

15.3 GoMyCode checkpoint coverage

Checkpoint instruction	Where it is satisfied
Identify a real-world problem; short research or literature review	Project report Chapters 1–2
Define architecture (front-end, back-end, database, cloud) with diagrams and justified choices	Report Chapter 3; this PRD Sections 7–9
Build with clean coding, modularity, scalability	Layered server, permission matrix, tests (Batches 1–3)

Checkpoint instruction	Where it is satisfied
Integrate cloud deployment tools, CI and testing	GitHub Actions, Render Blueprint, Vercel, Vitest/Playwright
Deploy to a public cloud platform with a public link	Vercel + Render + Atlas (Batch 3)
Professional report of 2,500–3,500 words	Condensed submission report derived from the full project report
10–minute presentation or demo	`docs/DEMO-SCRIPT.md` and pitch outline (Batch 3)

16. Risks and Mitigations

Risk	Impact	Mitigation
Frontend contract ambiguities (mock shapes differ between screens)	Broken screens	Treat `handlers.js` as truth; Playwright click-through per role; record deviations as ADRs
Render free-tier cold start	Slow first load in demo	Longer first-request timeout; open site a minute before demo; jobs catch up on boot
Ephemeral disk on Render	Lost uploads	Cloudinary provider in production
Paystack test vs live	No real money	Test mode documented as a limitation
NextAuth token lifetime vs API token	Silent 401s	Both 24 h; 401 interceptor signs the user out cleanly
Time pressure	Scope creep	Three batches only; cosmetic follow-ups go to PROGRESS.md
Secrets exposure (credentials file in KIT folder)	Breach	Never copy into repo; rotate Atlas password after deployment

17. Open Questions

1. Final production domain for the API (`api.kidsintech.school`) and LMS (`lms.kidsintech.school`): DNS access needed.
2. Course fee amounts per track (mock data uses ₦15,000–₦25,000).
3. Native-speaker review of Hausa strings.
4. Which KIT email address will send mail, and its SMTP credentials.